

МДК.01.01 Разработка программных модулей (123 пары) – Экзамен

Разработка программных модулей – процесс создания самостоятельных частей программы (модулей), которые обеспечивают заданные функции обработки автономно от других модулей. Такой метод программирования называют модульным программированием.

Разработка модулей может быть:

Как часть процесса разработки программного обеспечения (ПО). В зависимости от проекта разработка модулей может вестись последовательно или параллельно, но всегда завершается их объединением в единую систему.

В процессе доработки, усовершенствования или модернизации ранее созданного ПО. Модульная конструкция программного обеспечения позволяет вносить изменения в отдельные модули, удалять некоторые из них или дополнять систему новыми модулями.

Этапы

Изучение и проверка спецификации модуля, выбор языка программирования.

Выбор алгоритма и структуры данных.

Программирование (кодирование) модуля.

Шлифовка текста модуля – редактирование имеющихся комментариев, добавление дополнительных комментариев.

Проверка модуля – проверяется логика работы модуля, отлаживается его работа.

Компиляция модуля.

УП.01.01 Учебная практика (36 пар) – Зачет

1. Платформа .NET Framework

.NET Framework – это программная платформа, разработанная компанией Microsoft для создания и выполнения приложений под Windows.

Её главная цель – обеспечить единое, безопасное, многоплатформенное (в пределах экосистемы Microsoft) и удобное окружение для разработки приложений, независимо от используемого языка программирования.

.NET Framework выполняет несколько ключевых задач:

обеспечивает работу приложений на разных языках (C#, VB.NET, F#);

управляет памятью и безопасностью;

предоставляет богатый набор библиотек и API для типовых задач – от работы с файлами и сетью до графического интерфейса.

2. CLR (Common Language Runtime)

CLR – это основа .NET Framework, виртуальная машина, аналогичная JVM в Java.

Она выполняет байт– код промежуточного уровня (IL – Intermediate Language), а не исходный код напрямую.

Основные функции CLR:

JIT– компиляция (Just– In– Time) – преобразует IL– код в машинный код операционной системы при запуске программы.

Управление памятью – сборщик мусора (Garbage Collector) автоматически очищает неиспользуемые объекты.

Безопасность и проверка типов – предотвращает выполнение несанкционированного или небезопасного кода.

Обработка исключений – единый механизм для перехвата и обработки ошибок.

Многопоточность – встроенная поддержка потоков и синхронизации.

Таким образом, CLR создаёт управляемую (managed) среду выполнения, освобождая программиста от большинства низкоуровневых задач.

3. BCL (Base Class Library)

BCL – это базовая библиотека классов .NET Framework, ядро всей экосистемы.

Она содержит основные системные типы и функциональность, нужные любому приложению:

Примитивные типы данных (System.Int32, System.String, System.Boolean и др.);

Работа с файлами и потоками (System.IO);

Коллекции и структуры данных (System.Collections, System.Collections.Generic);

Работа с датами, временем и математикой (System.DateTime, System.Math);

Обработка ошибок (System.Exception);

Ввод– вывод, взаимодействие с консолью и сетью.

Благодаря BCL программисту не нужно реализовывать базовые механизмы – всё уже предоставлено в виде готовых классов и методов.

4. FCL (Framework Class Library)

FCL – это расширение BCL, включающее в себя дополнительные библиотеки, предназначенные для разработки более сложных приложений.

В неё входят компоненты для:

Разработки графических интерфейсов (Windows Forms, WPF);

Веб– разработки (ASP.NET);

Доступа к базам данных (ADO.NET);

Сетевого взаимодействия и веб– служб (System.Net, WCF, Web API);

Сериализации, XML/JSON, LINQ и других современных технологий.

Можно сказать, что BCL – это ядро, а FCL – это весь остальной функционал, надстроенный над ним.

5. Язык программирования C#

C# – это современный, объектно– ориентированный язык, разработанный специально для .NET Framework (а позже и .NET Core/.NET 5+).

Он сочетает в себе простоту синтаксиса, как у Java, и мощные возможности, например:

Поддержка принципов ООП (инкапсуляция, наследование, полиморфизм);

Работа с обобщениями (Generics);

Управление событиями и делегатами;

Асинхронное программирование с async/await;

Интероперабельность с неуправляемым кодом (через P/Invoke);

Поддержка LINQ, лямбда– выражений и безопасной работы с памятью.

Повторение ОАиП

Задание 1: Система управления банковским счётом (ООП, инкапсуляция, исключения)

Описание:

Разработайте класс BankAccount, который моделирует банковский счёт с функциями пополнения, снятия средств и проверки баланса. Система должна обрабатывать различные ошибки.

Требования:

Создайте класс BankAccount с приватными полями:

- accountNumber (строка)
- balance (decimal)
- ownerName (строка)

Реализуйте свойства с автоматической реализацией для чтения информации об счёте

Создайте методы:

- Deposit(decimal amount) – пополнение счёта (выбросить исключение, если сумма ≤ 0)
- Withdraw(decimal amount) – снятие денег (выбросить исключение, если недостаточно средств или сумма ≤ 0)
- GetBalance() – получить текущий баланс
- PrintStatement() – вывести выписку счёта

В методе Main():

- Создайте два счета
- Выполните несколько операций (пополнение, снятие)
- Обработайте исключения при попытке снять больше, чем есть
- Выведите выписки

Пример вывода:

Выписка счёта

Владелец: Иван Петров

Счёт: 12345

Баланс: 5000.00 руб.

Задание 2: Работа с массивами и коллекциями (методы, циклы)

Описание:

Создайте программу для анализа оценок студентов. Необходимо рассчитать средний балл, найти максимум/минимум, отфильтровать студентов по критериям.

Требования:

Создайте класс Student с полями:

- Name (имя)
- GPA (средний балл)
- Grades (массив оценок)

Создайте класс GradeAnalyzer со статическими методами:

- CalculateAverage(int[] grades) – средний балл массива
- FindMax(int[] grades) – максимальная оценка
- FindMin(int[] grades) – минимальная оценка
- FilterByGPA(Student[] students, double minGPA) – вернуть студентов с $GPA \geq minGPA$
- SortByGPA(Student[] students) – отсортировать по убыванию GPA (можно использовать Array.Sort с Comparison)

В методе Main():

- Создайте массив из 5 студентов с разными оценками
- Выведите информацию о каждом (имя, средний балл, оценки)
- Найдите студентов с $GPA \geq 4.0$
- Отсортируйте и выведите топ студентов

Пример вывода:

Все студенты

Алиса - GPA: 4.80

Боб - GPA: 3.50

Студенты с $GPA \geq 4.0$

Алиса - 4.80

Виктория - 4.30

Задание 3: Иерархия классов и полиморфизм (наследование, виртуальные методы, override)

Описание:

Создайте систему для работы с различными типами транспорта. Каждый вид имеет свои особенности движения и расхода топлива.

Требования:

Создайте базовый класс Vehicle с:

- Полями: Brand, FuelConsumption (литров на 100км)
- Виртуальным методом Move() – выводит "Транспорт движется"
- Методом CalculateFuelNeeded(double distance) – рассчитать нужное топливо

Создайте три класса-наследника:

- Car : Vehicle – переопределить Move() в "Автомобиль едет по дороге"
- Plane : Vehicle – переопределить Move() в "Самолет летит в небе"
- Boat : Vehicle – переопределить Move() в "Корабль плывет по воде"

Каждый класс должен иметь конструктор

В методе Main():

- Создайте массив Vehicle (содержащий объекты Car, Plane, Boat)
- Выполните полиморфный вызов Move() для каждого
- Рассчитайте необходимое топливо для маршрута в 1000км для каждого типа

Пример вывода:

Движение транспорта

Автомобиль едет по дороге

Самолет летит в небе

Корабль плывет по воде

Расход топлива на 1000 км

BMW (Car): 80.00 литров

Boeing 747 (Plane): 500.00 литров

Titanic (Boat): 200.00 литров

Задание 4: Работа с интерфейсами и абстрактными классами (интерфейсы, множественная реализация)

Описание:

Разработайте систему для работников различных категорий (менеджер, разработчик, дизайнер). Каждый сотрудник должен выполнять различные обязанности.

Требования:

Создайте интерфейсы:

- IWorker с методами: DoWork(), GetSalary()
- ITeamLead с методом: ManageTeam()
- IDeveloper с методом: WriteCode()

Создайте абстрактный класс Employee:

- Полями: Name, Salary, Department
- Абстрактным методом: PerformDuties()

Создайте конкретные классы:

- SoftwareDeveloper : Employee, IDeveloper
- Manager : Employee, IWorker, ITeamLead
- Designer : Employee, IWorker

В методе Main():

- Создайте коллекцию List<Employee> с разными работниками
- Выведите информацию о каждом
- Проверьте интерфейсы через is и as
- Выполните операции, специфичные для каждого типа

Пример вывода:

Сотрудники компании

Иван (разработчик, 80000 руб.)

- Пишет код на C#
- Может быть лидером команды

Мария (менеджер, 100000 руб.)

- Управляет командой
- Работает в отделе управления

Петр (дизайнер, 70000 руб.)

- Создает дизайн

Задание 5: Обработка исключений и работа со строками (enum, структуры, исключения, string методы)

Описание:

Создайте систему обработки заказов в ресторане с валидацией данных, обработкой ошибок и анализом информации.

Требования:

Создайте enum OrderStatus:

- Pending (в ожидании)
- Cooking (готовится)
- Ready (готово)
- Delivered (доставлено)
- Cancelled (отменено)

Создайте структуру MenuItem:

- Name (название блюда)
- Price (цена)
- Category (категория: "Салат", "Основное", "Напиток" и т.д.)

Создайте класс Order:

- OrderId (уникальный номер)
- CustomerName (имя клиента)
- Items (List<MenuItem>)
- Status (OrderStatus)
- CreatedDate (дата создания)

Методы:

- AddItem(MenuItem item) – добавить блюдо
- RemoveItem(string itemName) – удалить блюдо (выбросить исключение, если не найдено)
- GetTotalPrice() – получить общую сумму
- ChangeStatus(OrderStatus newStatus) – изменить статус
- PrintOrder() – вывести информацию о заказе
- GetItemsByCategory(string category) – получить блюда по категории

В методе Main():

- Создайте меню (List<MenuItem>)
- Создайте 2-3 заказа
- Добавьте блюда в заказы
- Изменяйте статусы
- Выполняйте поиск по категориям
- Обработайте исключения при удалении несуществующего блюда
- Выведите итоговую информацию

Пример вывода:

Меню

1. Цезарь (Салат) - 350 руб.
2. Паста Болоньезе (Основное) - 450 руб.
3. Апельсиновый сок (Напиток) - 150 руб.

Заказ #001

Клиент: Иван Петров

Статус: Готовится

Позиции:

- Паста Болоньезе (Основное) - 450 руб.

- Апельсиновый сок (Напиток) - 150 руб.

Итого: 600 руб.

Дата: 11.01.2026

Поиск блюд по категории "Салат"

- Цезарь - 350 руб.